FastAPI Reference

Static Files - `StaticFiles`

You can use the `StaticFiles` class to serve static files, like JavaScript, CSS, images, etc.

Read more about it in the FastAPI docs for Static Files.

You can import it directly from `fastapi.staticfiles`:

```
from fastapi.staticfiles import StaticFiles
```

class fastapi.staticfiles.StaticFiles

```
StaticFiles(
    *,
    directory=None,
    packages=None,
    html=False,
    check_dir=True,
    follow_symlink=False
)
```

Source code in `starlette/staticfiles.py`

```
def __init__(
    self,
    *,
    directory: PathLike | None = None,
    packages: list[str | tuple[str, str]] | None = None,
    html: bool = False,
    check_dir: bool = True,
    follow_symlink: bool = False,
) -> None:
    self.directory = directory
    self.packages = packages
    self.all_directories = self.get_directories(directory, packages)
    self.html = html
    self.config_checked = False
    self.follow_symlink = follow_symlink
    if check_dir and directory is not None and not os.path.isdir(directory):
        raise RuntimeError(f"Directory '{directory}' does not exist")
```

attr directory `instance-attribute`

```
directory = directory
```

attr packages `instance-attribute`

```
packages = packages
```

attr all_directories `instance-attribute`

Ask AI

```
all_directories = get_directories(directory, packages)
```

attr html `instance-attribute`

```
html = html
```

attr config_checked `instance-attribute`

```
config_checked = False
```

attr follow_symlink `instance-attribute`

```
follow_symlink = follow_symlink
```

meth get_directories

```
get_directories(directory=None, packages=None)
```

Given `directory` and `packages` arguments, return a list of all the directories that should be used for serving static files from.

Source code in `starlette/staticfiles.py`

```
def get_directories(
    self,
    directory: PathLike | None = None,
    packages: list[str | tuple[str, str]] | None = None,
) -> list[PathLike]:
    """
    Given `directory` and `packages` arguments, return a list of all the
    directories that should be used for serving static files from.
    """
    directories = []
    if directory is not None:
        directories.append(directory)

    for package in packages or []:
        if isinstance(package, tuple):
            package, statics_dir = package
        else:
            statics_dir = "statics"
        spec = importlib.util.find_spec(package)
        assert spec is not None, f"Package {package!r} could not be found."
        assert spec.origin is not None, f"Package {package!r} could not be found."
        package_directory = os.path.normpath(os.path.join(spec.origin, "..", statics_dir))
        assert os.path.isdir(package_directory), (
            f"Directory '{statics_dir!r}' in package {package!r} could not be found."
        )
        directories.append(package_directory)

    return directories
```

meth get_path

```
get_path(scope)
```

Given the ASGI scope, return the `path` string to serve up, with OS specific path separators, and any '..', '.' components removed.

Source code in `starlette/staticfiles.py`

```
def get_path(self, scope: Scope) -> str:
    """
    Given the ASGI scope, return the `path` string to serve up,
    with OS specific path separators, and any '..', '.' components removed.
    """
    route_path = get_route_path(scope)
    return os.path.normpath(os.path.join(*route_path.split("/")))
```

meth get_response `async`

```
get_response(path, scope)
```

Returns an HTTP response, given the incoming path, method and request headers.

Source code in `starlette/staticfiles.py`

```
async def get_response(self, path: str, scope: Scope) -> Response:
    """
    Returns an HTTP response, given the incoming path, method and request headers.
    """
    if scope["method"] not in ("GET", "HEAD"):
        raise HTTPException(status_code=405)

    try:
        full_path, stat_result = await anyio.to_thread.run_sync(self.lookup_path, path)
    except PermissionError:
        raise HTTPException(status_code=401)
    except OSError as exc:
        # Filename is too long, so it can't be a valid static file.
        if exc.errno == errno.ENAMETOOLONG:
            raise HTTPException(status_code=404)

        raise exc

    if stat_result and stat.S_ISREG(stat_result.st_mode):
        # We have a static file to serve.
        return self.file_response(full_path, stat_result, scope)

    elif stat_result and stat.S_ISDIR(stat_result.st_mode) and self.html:
        # We're in HTML mode, and have got a directory URL.
        # Check if we have 'index.html' file to serve.
        index_path = os.path.join(path, "index.html")
        full_path, stat_result = await anyio.to_thread.run_sync(self.lookup_path, index_path)
        if stat_result is not None and stat.S_ISREG(stat_result.st_mode):
            if not scope["path"].endswith("/"):
                # Directory URLs should redirect to always end in "/".
                url = URL(scope=scope)
                url = url.replace(path=url.path + "/")
                return RedirectResponse(url=url)
            return self.file_response(full_path, stat_result, scope)

    if self.html:
        # Check for '404.html' if we're in HTML mode.
        full_path, stat_result = await anyio.to_thread.run_sync(self.lookup_path, "404.html")
        if stat_result and stat.S_ISREG(stat_result.st_mode):
            return FileResponse(full_path, stat_result=stat_result, status_code=404)
    raise HTTPException(status_code=404)
```

meth lookup_path

```
lookup_path(path)
```

Source code in `starlette/staticfiles.py`

```python
def lookup_path(self, path: str) -> tuple[str, os.stat_result | None]:
    for directory in self.all_directories:
        joined_path = os.path.join(directory, path)
        if self.follow_symlink:
            full_path = os.path.abspath(joined_path)
            directory = os.path.abspath(directory)
        else:
            full_path = os.path.realpath(joined_path)
            directory = os.path.realpath(directory)
        if os.path.commonpath([full_path, directory]) != str(directory):
            # Don't allow misbehaving clients to break out of the static files directory.
            continue
        try:
            return full_path, os.stat(full_path)
        except (FileNotFoundError, NotADirectoryError):
            continue
    return "", None
```

`meth` file_response

```python
file_response(
    full_path, stat_result, scope, status_code=200
)
```

Source code in `starlette/staticfiles.py`

```python
def file_response(
    self,
    full_path: PathLike,
    stat_result: os.stat_result,
    scope: Scope,
    status_code: int = 200,
) -> Response:
    request_headers = Headers(scope=scope)

    response = FileResponse(full_path, status_code=status_code, stat_result=stat_result)
    if self.is_not_modified(response.headers, request_headers):
        return NotModifiedResponse(response.headers)
    return response
```

`meth` check_config `async`

```python
check_config()
```

Perform a one-off configuration check that StaticFiles is actually pointed at a directory, so that we can raise loud errors rather than just returning `4 0 4` responses.

Source code in `starlette/staticfiles.py`

```
183 async def check_config(self) -> None:
184     """
185     Perform a one-off configuration check that StaticFiles is actually
186     pointed at a directory, so that we can raise loud errors rather than
187     just returning 404 responses.
188     """
189     if self.directory is None:
190         return
191
192     try:
193         stat_result = await anyio.to_thread.run_sync(os.stat, self.directory)
194     except FileNotFoundError:
195         raise RuntimeError(f"StaticFiles directory '{self.directory}' does not exist.")
196     if not (stat.S_ISDIR(stat_result.st_mode) or stat.S_ISLNK(stat_result.st_mode)):
197         raise RuntimeError(f"StaticFiles path '{self.directory}' is not a directory.")
```

`meth` is_not_modified

```
is_not_modified(response_headers, request_headers)
```

Given the request and response headers, return `True` if an HTTP "Not Modified" response could be returned instead.

Source code in `starlette/staticfiles.py`

```
199 def is_not_modified(self, response_headers: Headers, request_headers: Headers) -> bool:
200     """
201     Given the request and response headers, return `True` if an HTTP
202     "Not Modified" response could be returned instead.
203     """
204     if if_none_match := request_headers.get("if-none-match"):
205         # The "etag" header is added by FileResponse, so it's always present.
206         etag = response_headers["etag"]
207         return etag in [tag.strip(" W/") for tag in if_none_match.split(",")]
208
209     try:
210         if_modified_since = parsedate(request_headers["if-modified-since"])
211         last_modified = parsedate(response_headers["last-modified"])
212         if if_modified_since is not None and last_modified is not None and if_modified_since >=
213 last_modified:
214             return True
215     except KeyError:
216         pass
217
        return False
```